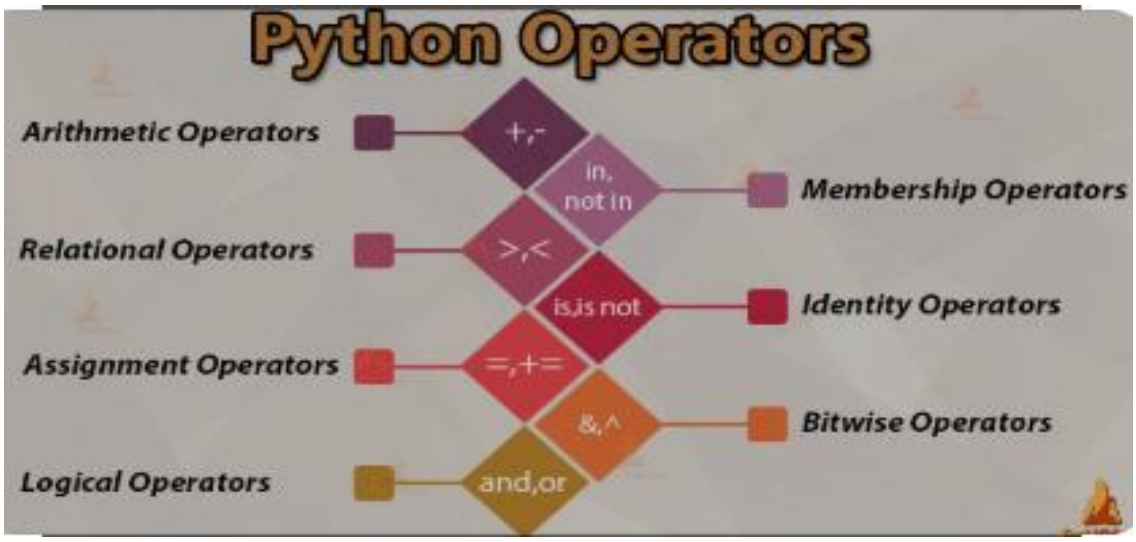
Operators in Python

Operators:

Operators are special symbols in Python that perform operations on variables and values. They are essential for performing calculations, comparisons, and logical operations in your code. Python supports a wide range of operators, which can be categorized into the following types:



---

1. Arithmetic Operators

Arithmetic operators are used to perform mathematical operations like addition, subtraction, multiplication, etc.

Operator	Description	Example
+	Addition	10 + 5 → 15
-	Subtraction	10 - 5 → 5
*	Multiplication	10 * 5 → 50
/	Division	10 / 5 → 2.0
%	Modulus (remainder)	10 % 3 → 1
**	Exponentiation (power)	2 ** 3 → 8
//	Floor Division	10 // 3 → 3

Example:

```python
a = 10
b = 3
print(a + b)  # 13
print(a - b)  # 7
print(a * b)  # 30
print(a / b)  # 3.333...
print(a % b)  # 1
print(a ** b) # 1000
print(a // b) # 3
```

---

2. Comparison Operators

Comparison operators are used to compare two values. They return a Boolean result (True or False).

Operator	Description	Example
==	Equal to	10 == 5 → False
!=	Not equal to	10 != 5 → True
>	Greater than	10 > 5 → True
<	Less than	10 < 5 → False
>=	Greater than or equal to	10 >= 5 → True
<=	Less than or equal to	10 <= 5 → False

Example:

```python
x = 10
y = 5
print(x == y)  # False
print(x != y)  # True
print(x > y)   # True
print(x < y)   # False
print(x >= y)  # True
print(x <= y)  # False
```

---

3. Logical Operators

Logical operators are used to combine conditional statements. They return a Boolean result (True or False).

Operator	Description	Example
and	True if both conditions are True	(10 > 5) and (5 < 3) → False
or	True if at least one condition is True	(10 > 5) or (5 < 3) → True
not	Reverses the result of the condition	not(10 > 5) → False

Example:

```python
a = 10
b = 5
c = 3
print(a > b and b > c)  # True
print(a > b or b < c)   # True
print(not (a > b))      # False
```

---

4. Assignment Operators

Assignment operators are used to assign values to variables. They can also perform operations while assigning.

Operator	Description	Example
=	Assign	x = 10
+=	Add and assign	x += 5 → x = x + 5
-=	Subtract and assign	x -= 5 → x = x - 5
*=	Multiply and assign	x *= 5 → x = x * 5
/=	Divide and assign	x /= 5 → x = x / 5

Operator	Description	Example
%=	Modulus and assign	x %= 5 → x = x % 5
**=	Exponent and assign	x **= 2 → x = x ** 2
//=	Floor divide and assign	x //= 3 → x = x // 3

Example:

```python
x = 10
x += 5  # x = x + 5 → 15
x -= 3  # x = x - 3 → 12
x *= 2  # x = x * 2 → 24
x /= 4  # x = x / 4 → 6.0
x %= 5  # x = x % 5 → 1.0
x **= 3 # x = x ** 3 → 1.0
x //= 2 # x = x // 2 → 0.0
```

---

5. Bitwise Operators

Bitwise operators perform operations on binary representations of integers.

Operator	Description	Example
&	Bitwise AND	10 & 5 → 0
`	`	Bitwise OR `105→15`
^	Bitwise XOR	10 ^ 5 → 15
~	Bitwise NOT (complement)	~10 → -11
<<	Left shift	10 << 2 → 40
>>	Right shift	10 >> 2 → 2

Example:

```python
a = 10  # Binary: 1010
b = 5   # Binary: 0101
print(a & b)  # 0 (0000)
print(a | b)  # 15 (1111)
print(a ^ b)  # 15 (1111)
print(~a)     # -11 (Two's complement)
print(a << 2) # 40 (101000)
print(a >> 2) # 2 (0010)
```

---

6. Membership Operators

Membership operators are used to check if a value exists in a sequence (e.g., list, string, tuple).

Operator	Description	Example
in	True if value exists in sequence	5 in [1, 2, 3, 4, 5] → True
not in	True if value does not exist in sequence	6 not in [1, 2, 3, 4, 5] → True

Example:

```python
my_list = [1, 2, 3, 4, 5]
print(3 in my_list)     # True
print(6 not in my_list) # True
```

---

7. Identity Operators

Identity operators are used to compare the memory locations of two objects.

Operator	Description	Example
is	True if both objects are the same	x is y → True if x and y point to the same object

Operator	Description	Example
is not	True if both objects are not the same	x is not y → True if x and y do not point to the same object

Example:

```python
x = 10
y = 10
z = [1, 2, 3]
w = [1, 2, 3]
print(x is y)      # True (same value, same memory location for small integers)
print(z is w)      # False (different memory locations)
print(x is not z)  # True
```

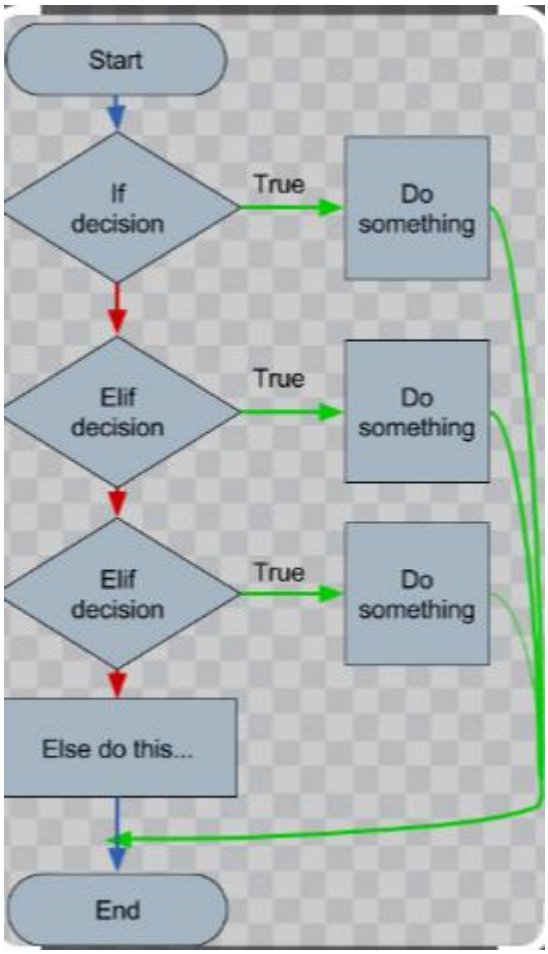
Summary of Operators

Category	Operators
Arithmetic	+, -, *, /, %, **, //
Comparison	==, !=, >, <, >=, <=
Logical	and, or, not
Assignment	=, +=, -=, *=, /=, %=, **=, //=
Bitwise	&, ` , ^, ~, <<, >>`
Membership	in, not in
Identity	is, is not

Flow Control: if, elif, else

Flow Control: if, elif, else

Flow control in Python allows you to execute code conditionally based on specific conditions. The if, elif, and else statements are used to control the flow of your program by evaluating conditions and executing blocks of code accordingly.



1.What is if Statement

Definition:

The if statement is used to execute a block of code **only if a specified condition is True**.

Syntax:

```python
if condition:
    # Code to execute if the condition is True
```

Example:

```python
age = 18
if age >= 18:
    print("You are eligible to vote.")
```

Explanation:

- The condition age >= 18 is evaluated.
- If the condition is True, the code inside the if block is executed.
- If the condition is False, the code inside the if block is skipped.

2.What is if-else Statement

Definition:

The if-else statement is used to execute one block of code if the condition is True and another block of code if the condition is False.

Syntax:

```python
if condition:
    # Code to execute if the condition is True
else:
    # Code to execute if the condition is False
```

Example:

```python
age = 16
if age >= 18:
    print("You are eligible to vote.")
else:
    print("You are not eligible to vote.")
```

Explanation:

- The condition age >= 18 is evaluated.
- If the condition is True, the code inside the if block is executed.
- If the condition is False, the code inside the else block is executed.

3.What is if-elif-else Statement

Definition:

The if-elif-else statement is used to check multiple conditions. It allows you to test multiple conditions in sequence and execute the block of code corresponding to the first True condition.

Syntax:

```python
if condition1:
    # Code to execute if condition1 is True
elif condition2:
    # Code to execute if condition2 is True
else:
    # Code to execute if all conditions are False
```

Example:

```python
score = 85
if score >= 90:
    print("Grade: A")
elif score >= 80:
    print("Grade: B")
elif score >= 70:
    print("Grade: C")
else:
    print("Grade: D")
```

Explanation:

- The conditions are evaluated in order:
    1. If score >= 90 is True, "Grade: A" is printed.
    2. If score >= 90 is False, the next condition score >= 80 is checked.
    3. If score >= 80 is True, "Grade: B" is printed.
    4. If both conditions are False, the else block is executed, and "Grade: D" is printed.

---

4. What is Nested if Statements

Definition:

Nested if statements are if statements inside other if statements. They are used to check multiple conditions in a hierarchical manner.

Syntax:

```python
if condition1:
    if condition2:
        # Code to execute if both condition1 and condition2 are True
```

Example:

```python
age = 20
has_id = True

if age >= 18:
    if has_id:
        print("You are eligible to enter.")
    else:
        print("You need an ID to enter.")
else:
    print("You are not eligible to enter.")
```

Explanation:

- The outer if checks if age >= 18.
    - If True, the inner if checks if has_id is True.
        - If both conditions are True, "You are eligible to enter." is printed.
        - If has_id is False, "You need an ID to enter." is printed.
- If the outer if condition is False, "You are not eligible to enter." is printed.

---

5. Ternary Operator (Conditional Expression)

Definition:

The ternary operator is a concise way to write an if-else statement in a single line.

Syntax:

```python
value_if_true if condition else value_if_false
```

Example:

```python
age = 20
status = "Eligible" if age >= 18 else "Not Eligible"
print(status)  # Output: Eligible
```

Explanation:

- The condition age >= 18 is evaluated.
- If True, the value "Eligible" is assigned to status.
- If False, the value "Not Eligible" is assigned to status.

---

6. Common Use Cases

Checking Multiple Conditions:

```python
x = 10
y = 20
z = 30

if x > y and x > z:
    print("x is the largest.")
elif y > x and y > z:
    print("y is the largest.")
else:
    print("z is the largest.")
```

Handling User Input:

```python
user_input = input("Enter 'yes' or 'no': ")

if user_input == "yes":
    print("You entered 'yes'.")
elif user_input == "no":
    print("You entered 'no'.")
else:
    print("Invalid input.")
```

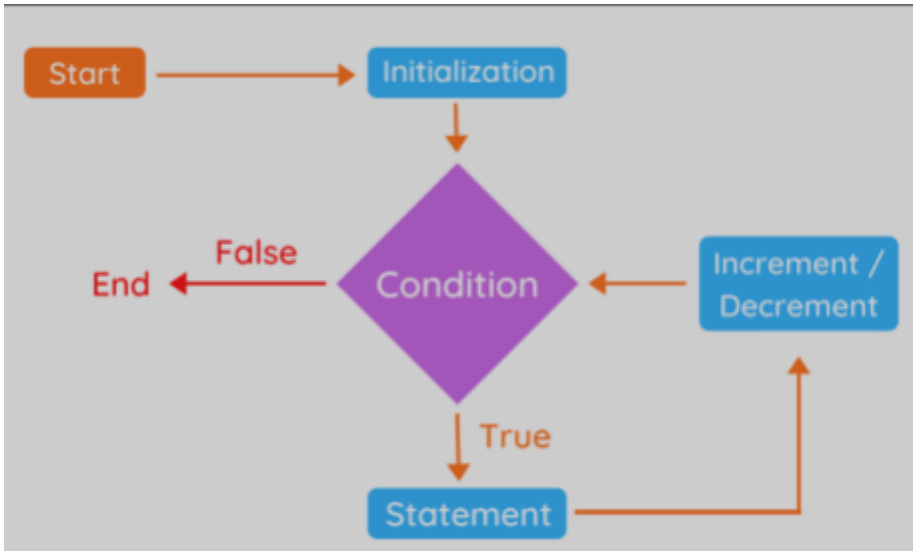
Summary of Flow Control

Statement	Description
if	Executes a block of code if the condition is True.
if-else	Executes one block if the condition is True, otherwise executes another.
if-elif-else	Checks multiple conditions and executes the block corresponding to the first True condition.
Nested if	if statements inside other if statements for hierarchical conditions.
Ternary Operator	A concise way to write an if-else statement in a single line.

Loops in Python

Loops:

Loops are a fundamental concept in programming that allow you to **repeat a block of code multiple times**. They are used to automate repetitive tasks, iterate over sequences (like lists, strings, or ranges), and perform operations until a specific condition is met. Python provides two main types of loops:



1. **for Loop**
2. **while Loop**

---

1. for Loop

Definition:

The for loop is used to **iterate over a sequence** (such as a list, tuple, string, or range) and execute a block of code for each item in the sequence.

Syntax:

```python
for item in sequence:
    # Code to execute for each item
```

Example:

```python
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)
```

Output:

```
apple
banana
cherry
```

Explanation:

- The for loop iterates over each item in the fruits list.
- For each iteration, the variable fruit takes the value of the current item, and the code inside the loop is executed.

Using range() with for Loop:

The range () function generates a sequence of numbers, which is often used with for loops.

Example:

```python
for i in range(5):  # Generates numbers from 0 to 4
    print(i)
```

Output:

```
0
1
2
3
4
```

Explanation:

- range(5) generates numbers from 0 to 4.
- The loop runs 5 times, with i taking values from 0 to 4.

Nested for Loops:

You can use a for loop inside another for loop to handle nested iterations.

Example:

```python
for i in range(3):  # Outer loop
    for j in range(2):  # Inner loop
        print(f"i = {i}, j = {j}")
```

Output:

```
i = 0, j = 0
i = 0, j = 1
i = 1, j = 0
i = 1, j = 1
i = 2, j = 0
i = 2, j = 1
```

Explanation:

- The outer loop runs 3 times (i = 0, 1, 2).
- For each iteration of the outer loop, the inner loop runs 2 times (j = 0, 1).

---

2. while Loop

Definition:

The while loop is used to **repeat a block of code as long as a condition is True**. It is useful when you don't know the number of iterations in advance.

Syntax:

```python
while condition:
    # Code to execute while the condition is True
```

Example:

```python
count = 0
while count < 5:
    print(count)
    count += 1  # Increment count by 1
```

Output:

```
0
1
2
3
4
```

Explanation:

- The loop continues to execute as long as the condition count < 5 is True.
- The variable count is incremented by 1 in each iteration.
- When count becomes 5, the condition becomes False, and the loop stops.

Infinite while Loop:

If the condition in a while loop is always True, the loop will run indefinitely. This is called an **infinite loop**.

Example:

```python
while True:
    print("This is an infinite loop!")
```

Explanation:

- The condition True is always True, so the loop never stops.

- To stop an infinite loop, you can use the break statement (explained below).

---

3. Loop Control Statements

Python provides special statements to control the flow of loops:

break Statement:

- The break statement is used to **exit the loop immediately**, even if the loop condition is still True.

Example:

```python
for i in range(10):
    if i == 5:
        break  # Exit the loop when i is 5
    print(i)
```

Output:

```
0
1
2
3
4
```

Explanation:

- The loop stops when i equals 5.

---

continue Statement:

- The continue statement is used to **skip the rest of the code in the current iteration** and move to the next iteration.

Example:

```python
for i in range(5):
    if i == 2:
        continue  # Skip the rest of the code for i = 2
    print(i)
```

Output:

```
0
1
3
4
```

Explanation:

- The iteration for i = 2 is skipped.

---

else Clause in Loops:

- The else clause in a loop is executed **when the loop finishes normally** (i.e., without encountering a break statement).

Example:

```python
for i in range(3):
    print(i)
else:
    print("Loop finished!")
```

Output:

```
0
1
2
Loop finished!
```

Explanation:

- The else block is executed after the loop completes all iterations.

---

4. When to Use for vs while Loops

for Loop	while Loop
Used when the number of iterations is known.	Used when the number of iterations is unknown.
Iterates over a sequence (e.g., list, string, range).	Repeats as long as a condition is True.
Example: Iterating over a list of items.	Example: Reading user input until a valid response is given.

---

5. Practical Examples

Example 1: Sum of Numbers in a List

```python
numbers = [1, 2, 3, 4, 5]
total = 0
for num in numbers:
    total += num
print("Sum:", total)  # Output: Sum: 15
```

Example 2: Finding Factorial Using while Loop

```python
n = 5
factorial = 1
while n > 0:
    factorial *= n
    n -= 1
print("Factorial:", factorial)  # Output: Factorial: 120
```

Example 3: Password Validation

```python
password = ""
while password != "secret":
    password = input("Enter the password: ")
print("Access granted!")
```

Summary of Loops

Loop Type	Description
for Loop	Iterates over a sequence (e.g., list, string, range).
while Loop	Repeats a block of code as long as a condition is True.
break	Exits the loop immediately.
continue	Skips the rest of the code in the current iteration.
else Clause	Executes when the loop finishes normally (without a break).

Python Identifiers

Identifiers:

In Python, an **identifier** is a name given to variables, functions, classes, modules, or other objects. Identifiers are used to uniquely identify these objects in your code. Understanding the rules and conventions for naming identifiers is crucial for writing clean, readable, and error-free Python programs.

1. Rules for Naming Identifiers

Python has specific rules for naming identifiers. Violating these rules will result in a **Syntax Error**.

Rules:

1. **Allowed Characters**:
    - Identifiers can contain:
        - Letters (a-z, A-Z)
        - Digits (0-9)
        - Underscores (_)
    - Example: my_var, var1, _name
2. **First Character**:
    - The first character of an identifier **cannot be a digit**.
    - Example:
        - Valid: name, _age

        - Invalid: 1name, 2var

3. **Case Sensitivity**:
    - Identifiers are **case-sensitive**.
    - Example: myVar and myvar are two different identifiers.
4. **Reserved Keywords**:
    - Identifiers cannot be the same as Python's **reserved keywords** (e.g., if, else, for, while, etc.).
    - Example:
        - Valid: my_if, while_loop
        - Invalid: if, while
5. **Length**:
    - There is no limit to the length of an identifier, but it is recommended to keep it concise and meaningful.

2. Examples of Valid and Invalid Identifiers

Valid Identifiers:

```python
name = "Alice"
age = 25
user_name = "john_doe"
_total = 100
PI = 3.14
```

Invalid Identifiers:

```python
2name = "Bob"         # Starts with a digit
my-name = "Alice"     # Contains a hyphen (-)
class = "Python"      # Reserved keyword
```

3. Best Practices for Naming Identifiers

While Python allows flexibility in naming identifiers, following best practices ensures that your code is readable and maintainable.

1. Use Descriptive Names:

- Choose names that describe the purpose of the variable, function, or class.
- Example:

```python
# Good
student_name = "Alice"
total_score = 95

# Bad
a = "Alice"
b = 95
```

2. Use Snake Case for Variables and Functions:

- In Python, it is conventional to use **snake_case** (lowercase letters with underscores) for variable and function names.
- Example:

```python
user_name = "John"
def calculate_total():
    pass
```

3. Use Pascal Case for Class Names:

- For class names, use **PascalCase** (capitalize the first letter of each word).
- Example:

```python
class StudentDetails:
    pass
```

4. Avoid Single-Letter Names:

- Single-letter names (e.g., x, y, z) are acceptable in small scopes (e.g., loop counters), but avoid them for larger scopes.
- Example:

```python
# Acceptable
for i in range(10):
    print(i)

# Not recommended
x = 10
y = 20
```

5. Use UPPERCASE for Constants:

- Constants (values that do not change) are typically written in **UPPERCASE** with underscores.
- Example:

```python
PI = 3.14
MAX_USERS = 100
```

4. Reserved Keywords in Python

Python has a set of reserved keywords that cannot be used as identifiers. These keywords have special meanings and are used to define the syntax and structure of the language.

List of Python Keywords:

and	as	assert	async	await
break	class	continue	def	del

elif	else	except	finally	for
from	global	if	import	in
is	lambda	nonlocal	not	or
pass	raise	return	try	while
with	yield			

Example:

```python
# Invalid (using a reserved keyword as an identifier)
if = 10  # SyntaxError

# Valid (using a non-reserved keyword)
my_if = 10
```

5. Special Identifiers

1. _ (Single Underscore):

- A single underscore is often used as a placeholder for variables that are not needed.
- Example:

```python
for _ in range(5):
    print("Hello")
```

2. __ (Double Underscore):

- Identifiers starting with a double underscore (e.g., __name) are used for **name mangling** in classes to make them private.
- Example:

```python
class MyClass:
    def __init__(self):
        self.__private_var = 10
```

3. __name__ (Dunder/Magic Methods):

- Identifiers surrounded by double underscores (e.g., __init__, __name__) are called **dunder methods** or **magic methods**. They have special meanings in Python.
- Example:

```python
if __name__ == "__main__":
    print("This is the main module.")
```

6. Checking if a String is a Valid Identifier

You can use the str.isidentifier() method to check if a string is a valid identifier.

Example:

```python
print("my_var".isidentifier())  # True
print("2name".isidentifier())   # False
print("class".isidentifier())   # False (reserved keyword)
```

Summary of Python Identifiers

Aspect	Description
Allowed Characters	Letters, digits, and underscores.
First Character	Cannot be a digit.
Case Sensitivity	Identifiers are case-sensitive.

Aspect	Description
Reserved Keywords	Cannot use Python's reserved keywords as identifiers.
Best Practices	Use descriptive names, snake_case for variables, PascalCase for classes, etc.
Special Identifiers	_, __, and dunder methods (e.g., __name__) have special meanings.

Building Functions in Python

Functions:

Functions are reusable blocks of code that perform a specific task. Python provides several **built-in functions** that are readily available for use. In this section, we will explore the following built-in functions in detail:

1. **print()**
2. **type()**
3. **id()**
4. **sys**
5. **len()**

1. print() Function

Definition:

The print() function is used to **display output** to the console. It can print strings, numbers, variables, and even expressions.

Syntax:

```python
print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)
```

Parameters:

- *Objects: The values to print (can be multiple, separated by commas).

- sep: Separator between objects (default is a space ' ').
- end: Character to print at the end (default is a newline '\n').
- file: Where to print (default is sys.stdout, the console).
- flush: Whether to forcibly flush the stream (default is False).

Examples:

```python
# Basic usage
print("Hello, World!")  # Output: Hello, World!

# Printing multiple objects
print("Hello", "World", sep="-")  # Output: Hello-World

# Changing the end character
print("Hello", end="!!!\n")  # Output: Hello!!!

# Printing variables
name = "Alice"
age = 25
print("Name:", name, "Age:", age)  # Output: Name: Alice Age: 25
```

2. type() Function

Definition:

The type() function is used to **determine the data type** of an object. It is useful for debugging and ensuring that variables are of the expected type.

Syntax:

```python
type(object)
```

Parameters:

- object: The object whose type you want to check.

Examples:

```python
# Checking data types
print(type(10))          # Output: <class 'int'>
print(type(3.14))        # Output: <class 'float'>
print(type("Hello"))     # Output: <class 'str'>
print(type([1, 2, 3]))   # Output: <class 'list'>
print(type(True))        # Output: <class 'bool'>
```

3. id() Function

Definition:

The id() function returns the **unique identifier (memory address)** of an object. This identifier is unique and constant for the object during its lifetime.

Syntax:

```python
id(object)
```

Parameters:

- object: The object whose identifier you want to retrieve.

Examples:

```python
a = 10
b = 10
c = [1, 2, 3]

print(id(a))  # Output: Memory address of 'a' (e.g., 140736123456)
print(id(b))  # Output: Same as 'a' (small integers are reused in Python)
print(id(c))  # Output: Memory address of 'c' (e.g., 140736123456)
```

4. sys Module

Definition:

The sys module provides access to **system-specific parameters and functions**. It is part of Python's standard library and must be imported before use.

Common Uses:

- Accessing command-line arguments.
- Interacting with the Python interpreter.
- Managing system-specific configurations.

Syntax:

```python
import sys
```

Examples:

```python
import sys

# Accessing command-line arguments
print("Script name:", sys.argv[0])  # Output: Script name: script.py

# Exiting the program
if len(sys.argv) < 2:
    print("Error: Missing argument.")
    sys.exit(1)  # Exit with status code 1

# Python version
print("Python version:", sys.version)  # Output: Python version: 3.x.x
```

5. len() Function

Definition:

The len() function returns the **length** (number of items) of an object. It works with sequences like strings, lists, tuples, and dictionaries.

Syntax:

```python
len(object)
```

Parameters:

- object: The object whose length you want to calculate.

Examples:

```python
# Length of a string
print(len("Hello"))  # Output: 5

# Length of a list
print(len([1, 2, 3, 4]))  # Output: 4

# Length of a tuple
print(len((1, 2, 3)))  # Output: 3

# Length of a dictionary
print(len({"name": "Alice", "age": 25}))  # Output: 2
```

Summary of Functions

Function	**Description**
print()	Displays output to the console.
type()	Returns the data type of an object.
id()	Returns the unique identifier (memory address) of an object.
sys	Provides access to system-specific parameters and functions.
len()	Returns the length (number of items) of an object.

INTERVIEW QUESTIONS & ANSWERS

BASIC LEVEL QUESTIONS

Q1. What are the different types of operators in Python?

Ans:

1. Arithmetic Operators: +, -, *, /, //, %, **
2. Comparison Operators: ==, !=, >, <, >=, <=

3. Logical Operators: and, or, not
4. Bitwise Operators: &, |, ^, ~, <<, >>
5. Assignment Operators: =, +=, -=, *=, /=, etc.
6. Membership Operators: in, not in
7. Identity Operators: is, is not

Q2. What is the purpose of if, elif, and else in Python?

Ans**:** These are used for decision-making in Python programs.

Example:

```
x = 10

if x > 10:
   print("Greater")
elif x == 10:
   print("Equal")
else:
   print("Smaller")
```

Q3. What are the types of loops in Python?
Ans: There are two types of loops in python
1. for loop (used for iteration over sequences)
2. while loop (runs as long as a condition `is True)`

Q4. What is an identifier in Python?
Ans: An identifier is the name used to identify variables, functions, classes, etc.

Q5. What is the purpose of the print() function?
Ans: It is used to display output on the console.
Example:
```
print("Hello, World!")
```

INTERMEDIATE LEVEL QUESTIONS & ANSWERS
Q6. How do bitwise operators work in Python?

Ans:
Bitwise operators manipulate bits at a binary level.
Example:

```
a = 5  # 0b0101
b = 3  # 0b0011

print(a & b)  # 1  (0b0001 - AND)
print(a | b)  # 7  (0b0111 - OR)
print(a ^ b)  # 6  (0b0110 - XOR)
print(~a)     # -6 (inverts bits)
```

Q7. Can you write a nested if-else statement in Python?
Ans:
Yes. Example:

```
x = 10
if x > 0:
   if x % 2 == 0:
      print("Positive Even")
   else:
      print("Positive Odd")
else:
   print("Negative or Zero")
```

Q8. What is the difference between a for loop and a list comprehension?
Ans:
A list comprehension is a more concise way to write a loop for creating lists.

```
# Using for loop
squares = []
for x in range(5):
   squares.append(x**2)

# Using list comprehension
squares = [x**2 for x in range(5)]
```

Q9. What are reserved keywords in Python?
Ans:
Reserved words like if, for, while, class, def, import cannot be used as identifiers.
To check all keywords in Python:

```
import keyword
```

```
print(keyword.kwlist)
```

Q10. What is the purpose of the sys module?
Ans:
The sys module provides access to system-related functions and variables.

```
import sys
print(sys.version)  # Get Python version
print(sys.argv)  # Command-line arguments
```

ADVANCED LEVEL QUESTIONS & ANSWERS

Q11. What is operator overloading, and how is it implemented in Python?
Ans: Operator overloading allows defining custom behavior for built-in operators (+, -, *, etc.) using special methods.
Example:

```
class Vector:
    def __init__(self, x, y):
        self.x = x
        self.y = y

    def __add__(self, other):
        return Vector(self.x + other.x, self.y + other.y)

v1 = Vector(2, 3)
v2 = Vector(1, 5)
v3 = v1 + v2  # Uses __add__()
print(v3.x, v3.y)  # Output: 3, 8
```

Q12. What is the difference between match-case (Python 3.10+) and if-elif-else?
Ans:

- match-case is more **efficient** and works like **switch statements** in other languages.
- It supports **pattern matching**.

Example:

```
def check_value(x):
    match x:
        case 1:
            print("One")
```

```
    case 2 | 3:
        print("Two or Three")
    case _:
        print("Other")

check_value(2)  # Output: Two or Three
```

Q13. What is an iterator, and how does Python handle iteration?
Ans: An **iterator** is an object with __iter__() and __next__() methods.
Example:

```
class Counter:
    def __init__(self, start, end):
        self.current = start
        self.end = end

    def __iter__(self):
        return self

    def __next__(self):
        if self.current >= self.end:
            raise StopIteration
        self.current += 1
        return self.current - 1

c = Counter(1, 5)
for num in c:
    print(num)
```

Q14. What is name mangling in Python?
Ans:

- Python changes names of attributes prefixed with **double underscores (__)** to avoid name conflicts in subclasses.
- The name is transformed as _ClassName__attribute.

Example:
python
CopyEdit

```
class A:
    def __init__(self):
        self.__private = 10
```

```
a = A()
print(a.__private)  # AttributeError
print(a._A__private)  # Works (name mangling)
```

Q15. What is the difference between sys.exit() and os._exit()?
Answer:

- sys.exit() raises a SystemExit exception, allowing cleanup.
- os._exit() terminates the process immediately without cleanup.

Example:

```
import sys
sys.exit("Exiting...")  # Raises SystemExit

import os
os._exit(0) # Terminates process immediately
```